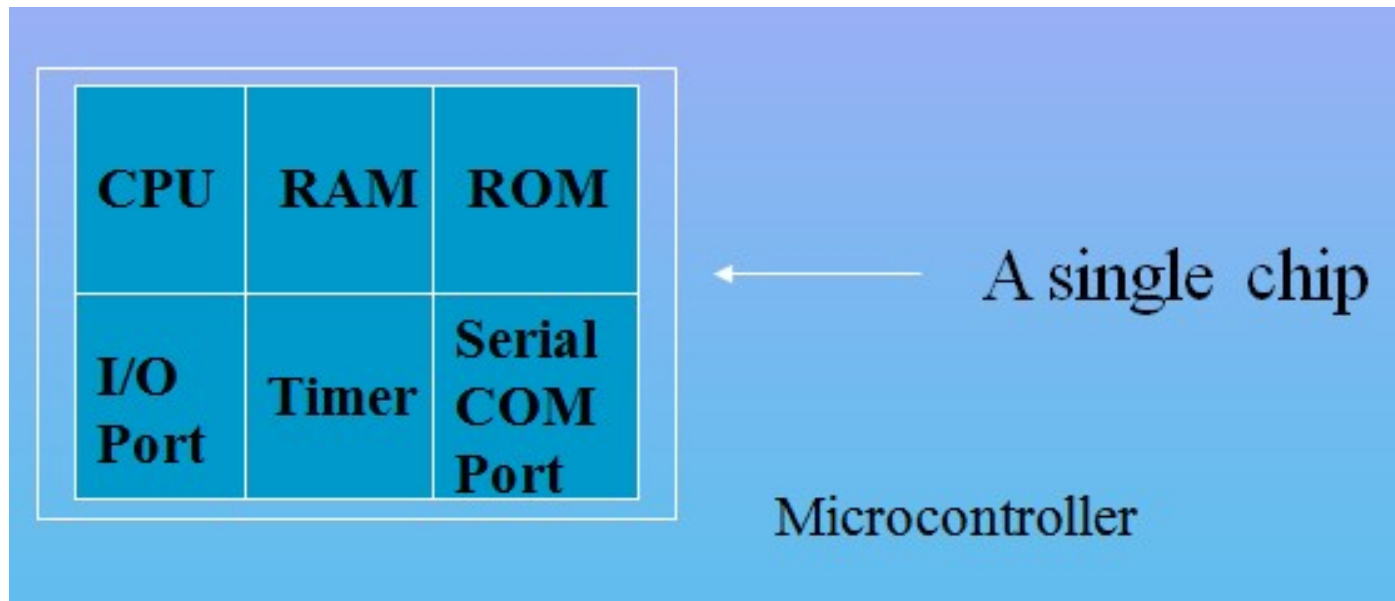
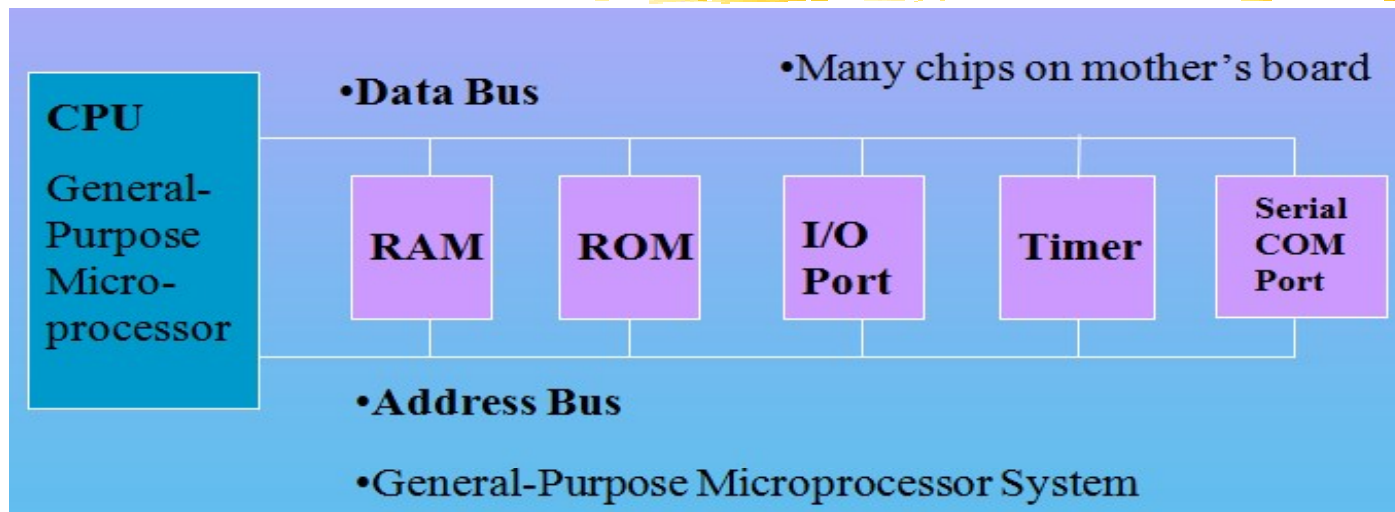


Unit 3

Embedded system and IOT Applications

-Presented By
Dr. Ashish Nainwal
Assistant Professor
ECE Dept.
FET GK(DU) Hridwar

Microprocessor vs. Microcontroller



Microprocessor vs. Microcontroller

Microprocessor

- ⌘ CPU is stand-alone, RAM, ROM, I/O, timer are separate
- ⌘ Designer can decide on the amount of ROM, RAM and I/O ports.
- ⌘ Expansive
- ⌘ Versatility
- ⌘ General-purpose

Microcontroller

- ⌘ CPU, RAM, ROM, I/O and timer are all on a single chip
- ⌘ Fixed amount of on-chip ROM, RAM, I/O ports.
- ⌘ Not Expansive
- ⌘ Single-purpose
- ⌘ Special Purpose.

Embedded Vs External Memory devices



- ⌘ A device has all the functional blocks on chip, including the program and data memory. (**Embedded Memory Device**)
- ⌘ There is no external data/address bus provided.
- ⌘ E.g. ATMEL 89C2051 has timer/counters, on chip RAM, EEPROM, I/Os, CPU unit. It has only 20 pins.
- ⌘ A device like MCS-51 family needs external program memory interface. (**External Memory Device**)
- ⌘ In this case part of program execution is possible from the on chip program memory and rest of the program execution is from the external memory.
- ⌘ This type of device required external data /address bus to access external memory so required additional PCB space.

CISC and RISC Processors

⌘ CISC is a Complex Instruction Set Computer. It is a computer that can address a large number of instructions.

⌘ **Example:** Intel x86 architecture, Motorola 68000

Example Instruction `ADD R1, [MEM]`

This instruction may **fetch data from memory and add it in one instruction.**

⌘ In the early 1980s, computer designers recommended that computers should use fewer instructions with simple constructs so that they can be executed much faster within the CPU without having to use memory. Such computers are classified as Reduced Instruction Set Computer or RISC.

⌘ **Examples:** ARM architecture

Example Instructions

`LOAD R1, MEM`

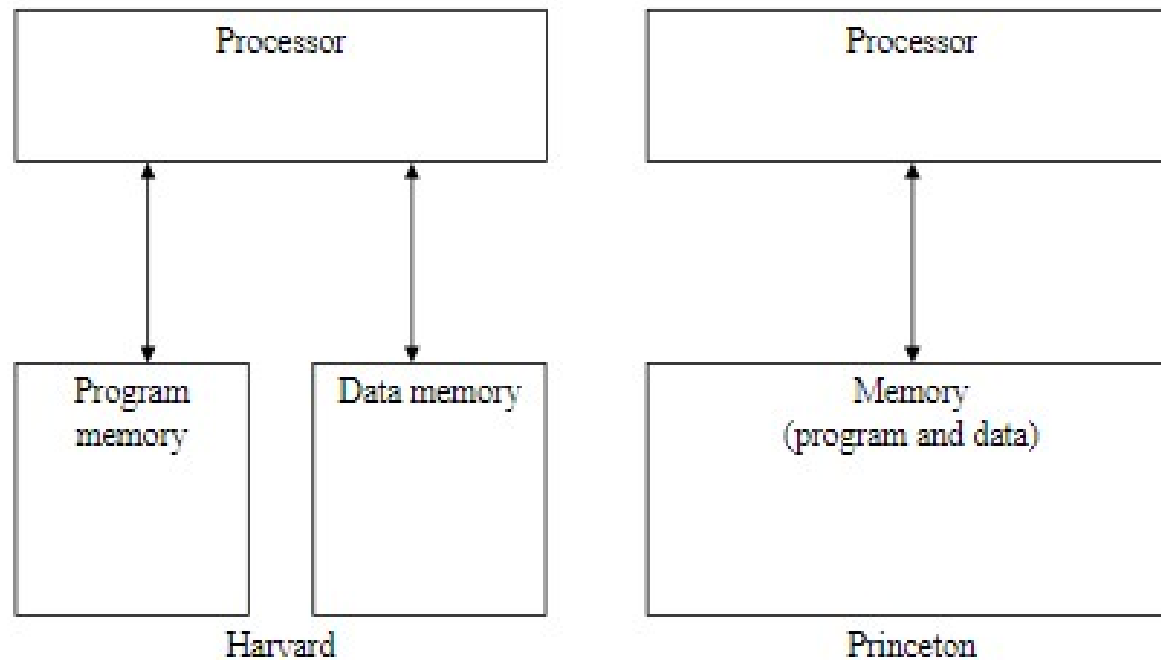
`ADD R1, R2`

`STORE R1, MEM`

⌘ The task is divided into **multiple simple instructions.**

	RISC	CISC
Instruction Set Complexity	Supports a smaller number of instructions that perform simpler operations.	Supports a wide variety of instructions for complex operations.
Instruction Format	Uses a fixed-length instruction format.	Can have variable-length instruction formats.
Pipeline Design	Has a simpler pipeline design.	Can have longer pipelines with more complex stages.
Memory Access	Relies on load/store architecture; memory access through load and store instructions.	Supports instructions that can directly operate on memory.
Performance and Optimization	Designed for executing instructions in a small number of clock cycles.	May require more clock cycles to execute instructions
Code Density	May require more instructions to perform a task, resulting in larger program sizes.	Can often perform more operations in a single instruction, leading to smaller program sizes.

Harvard and Von-Neumann (Princeton) Architecture



Harvard and Von-Neumann (Princeton) Architecture

Harvard Architecture	Von-Neumann Architecture
Separate buses for Instruction and Data fetching	Single shared bus for Instruction and Data fetching
Easier to Pipeline, so high performance can be achieved	Low performance Compared to Harvard Architecture
Comparatively high cost	Cheaper
No memory alignment problems	Allows self modifying code†
Since data memory and program memory are stored physically in different locations, no chances for accidental corruption of program memory	Since data memory and program memory are stored physically in same chip, chances for accidental corruption of program memory

μC based Embedded Systems



- ⌘ Special purpose computer system usually completely inside the device it controls
- ⌘ Has specific requirements and performs pre-defined tasks
- ⌘ Cost reduction compared to general purpose processor
- ⌘ Different design criteria
 - ☒ Performance
 - ☒ Reliability
 - ☒ Availability
 - ☒ Safety

8051 Microcontroller

- ⌘ Intel introduced 8051, referred as MCS-51, in 1981.
- ⌘ The 8051 is an **8-bit processor**
 - ⏏ The CPU can work on only 8 bits of data at a time
- ⌘ The 8051 became widely popular after allowing other manufactures to make and market any flavor of the 8051.

Features of 8051

8 bit Processor

4KB **Internal** ROM

128 Bytes **Internal** RAM

Four 8 BIT I/O PORTS (32 I/O LINES)

Two 16 Bit Timers/Counters

On Chip Full Duplex UART for Serial Communication

5 Vector Interrupts (2 External, 3 Internal - Timer0,Timer1,Serial)

Bit addressable

16 bit Address bus

64k External Code Memory

64k External Data Memory

16-bit program counter to access external Code Memory and

16 bit Data Pointer to access external Data Memory

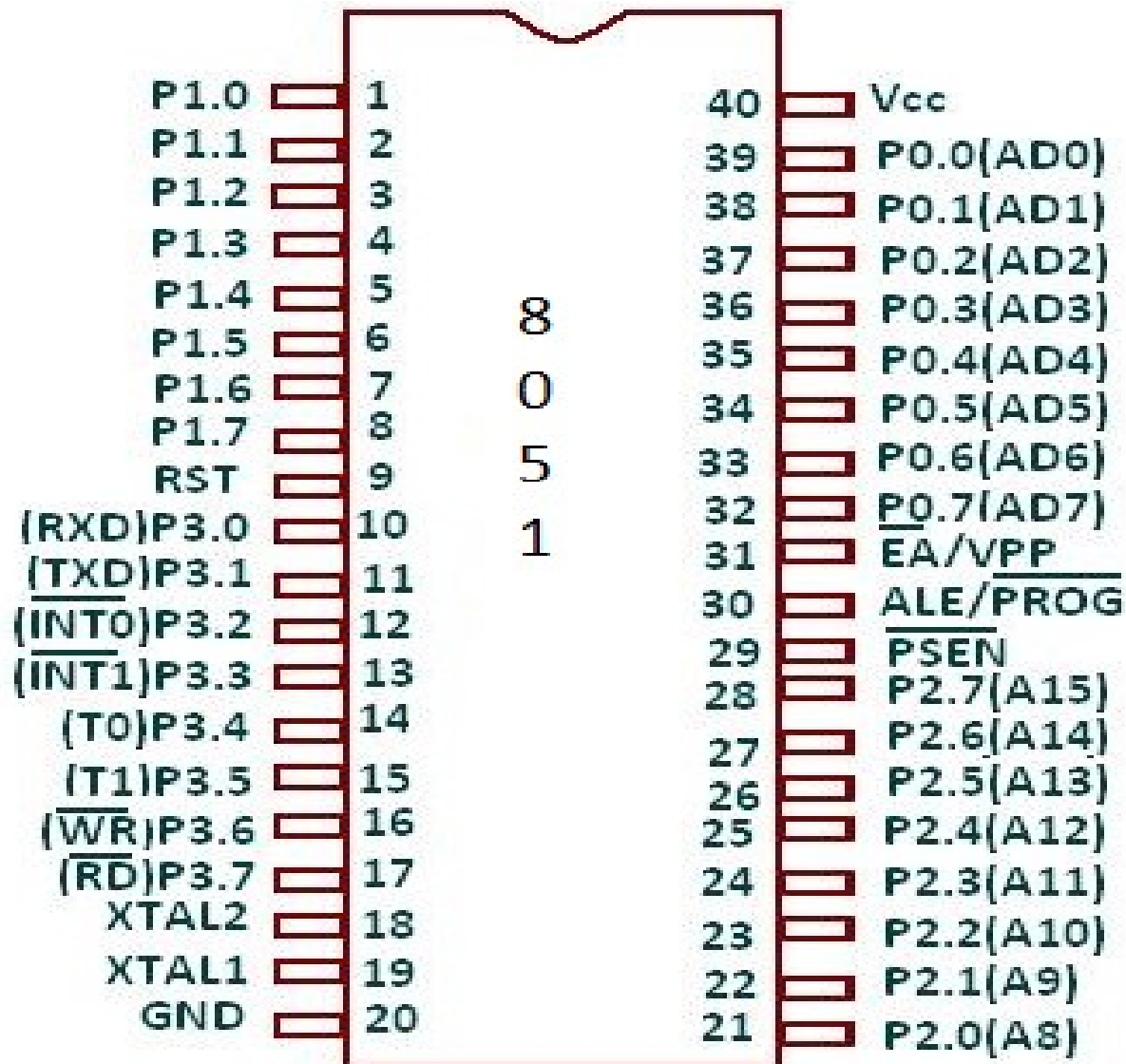
32 General Purpose Registers each of 8 bits

8051 Family

- ⌘ The 8052 is a subset of the 8051
- ⌘ The 8031 is a ROM-less 8051
 - ☒ Add external ROM to it
 - ☒ You lose two ports, and leave only 2 ports for I/O operations

Feature	8051	8052	8031
ROM (on-chip program space in bytes)	4K	8K	0K
RAM (bytes)	128	256	128
Timers	2	3	2
I/O pins	32	32	32
Serial port	1	1	1
Interrupt sources	6	8	6

Pin Diagram

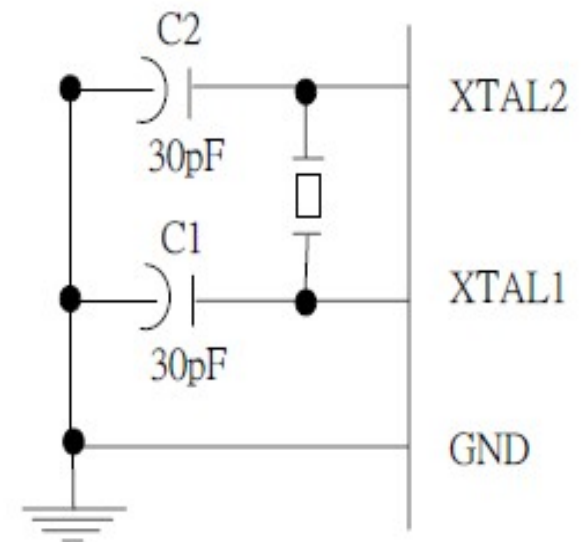


Pin Description of the 8051

- ⌘ 8051 family members (e.g., 8751, 89C51, 89C52, DS89C4x0)
 - ⌘ Have **40 pins** dedicated for various functions such as I/O, RD, WR, address, data, and interrupts.
 - ⌘ Come in different packages, such as
 - ⌘ *DIP(dual in-line package),*
 - ⌘ *QFP(quad flat package), and*
 - ⌘ *LLC(leadless chip carrier)*
- ⌘ Some companies provide a 20-pin version of the 8051 with a reduced number of I/O ports for less demanding applications

XTAL1 and XTAL2

- ⌘ The 8051 has an on-chip oscillator but requires an external crystal to run it
 - ☒ A quartz crystal oscillator is connected to inputs XTAL1 (pin19) and XTAL2 (pin18)
 - ☒ The quartz crystal oscillator also needs two capacitors of 30 pF value
 - ☒ The original 8051 operates at **12 MHZ**



RST

- ⌘ RESET pin is an input and is active high (normally low)
- ⌘ Upon applying a high pulse to this pin, the microcontroller will reset and terminate all activities
- ⌘ This is often referred to as a power-on reset
- ⌘ Activating a power-on reset will cause all values in the registers to be lost

RESET value of some
8051 registers

we must place
the first line of
source code in
ROM location 0

Register	Reset Value
PC	0000
DPTR	0000
ACC	00
PSW	00
SP	07
B	00
P0-P3	FF



⌘ PSEN (out): Program Store Enable, the read signal for external program memory (active low).

1-Execute internal program memory

0-Execute external Program memory

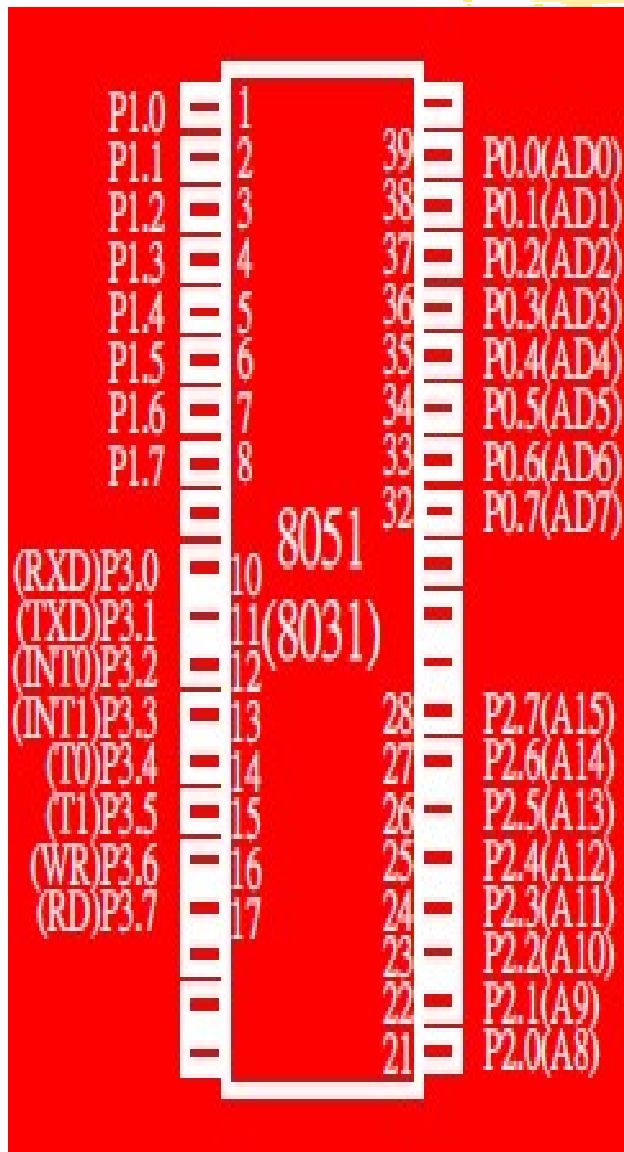
⌘ ALE (out): Address Latch Enable, to latch address outputs at Port0 and Port2

⌘ EA (in): External Access Enable, active low to access external program memory locations 0 to 4K

1- Fetch instruction from internal program memory then external.

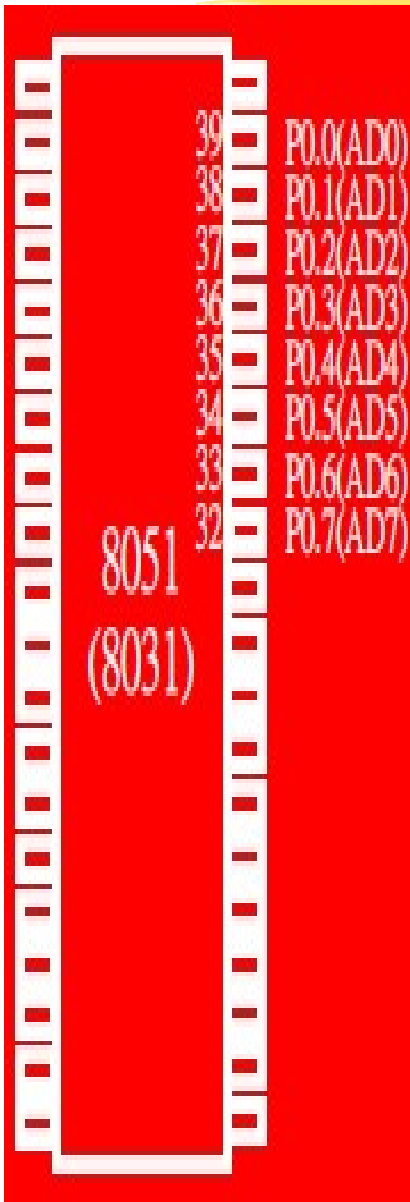
0-Fetch instruction from external memory.

I/O Port Pins



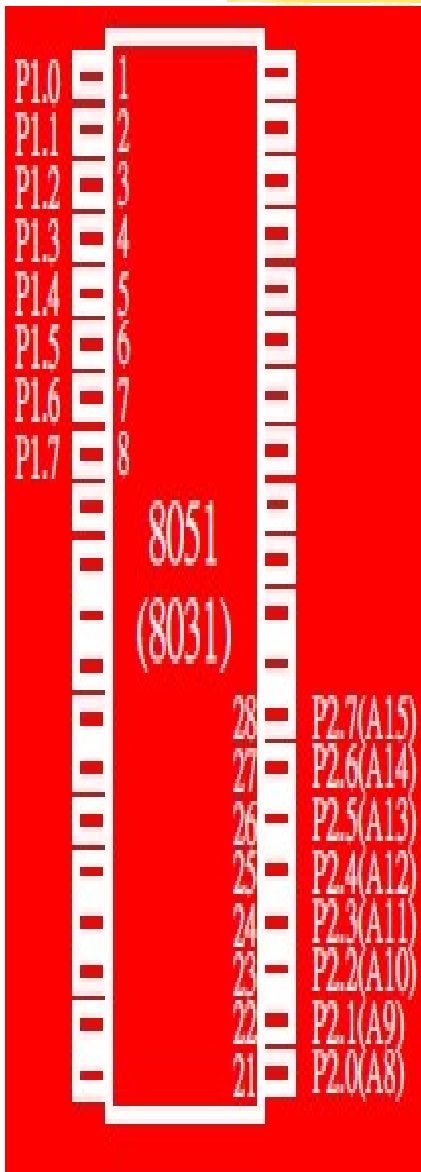
- ⌘ The four 8-bit I/O ports **P0**, **P1**, **P2** and **P3** each uses 8 pins.
- ⌘ All the ports upon RESET are configured as output, ready to be used as input ports by the external device.

Port 0



- ⌘ Port 0 is **also** designated as **AD0-AD7**.
- ⌘ When connecting an 8051 to an external memory, port 0 provides both address and data.
- ⌘ The 8051 multiplexes address and data through port 0 to save pins.
- ⌘ **ALE** indicates if P0 has address or data.
 - ☒ *When ALE=0, it provides data D0-D7*
 - ☒ *When ALE=1, it has address A0-A7*

Port 1 and Port 2

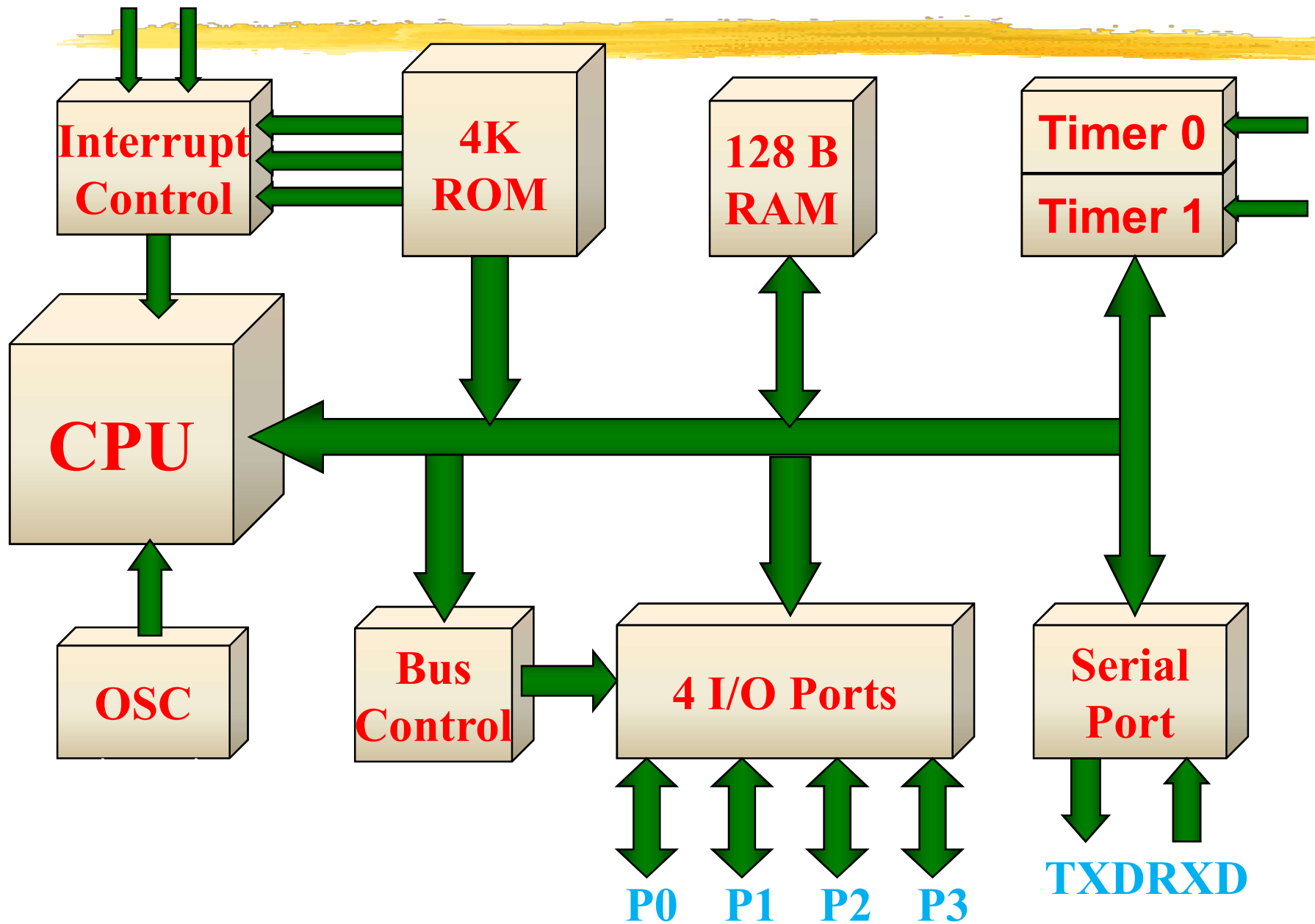


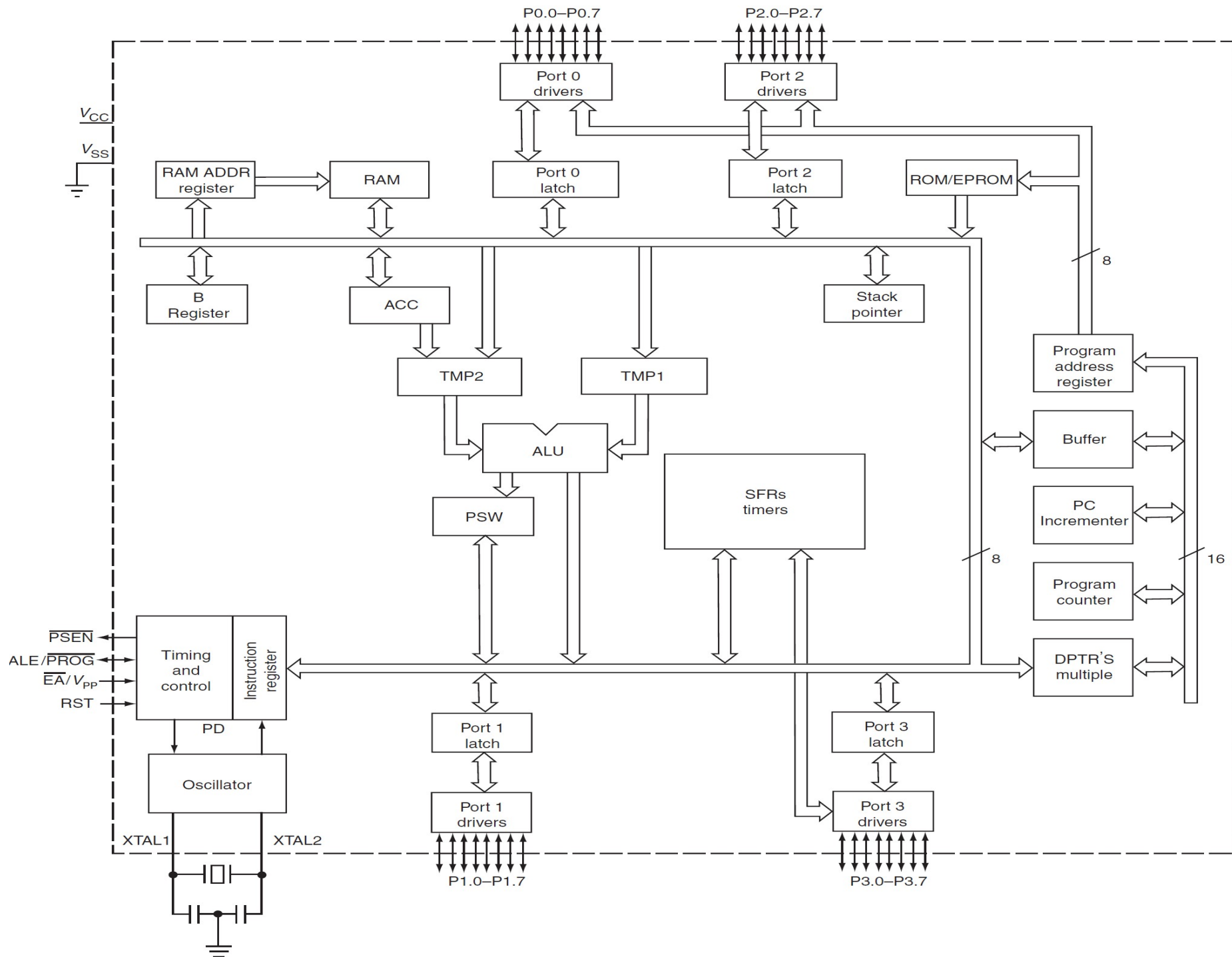
- ⌘ In 8051-based systems **with no external memory connection**:
 - ☒ Both P1 and P2 are used as simple I/O.
- ⌘ In 8051-based systems **with external memory connections**:
 - ☒ Port 2 must be used along with P0 to provide the 16-bit address for the external memory.
 - ☒ P0 provides the lower 8 bits via A0 - A7.
 - ☒ P2 is used for the upper 8 bits of the 16-bit address, designated as A8 - A15, and it cannot be used for I/O.

Port 3

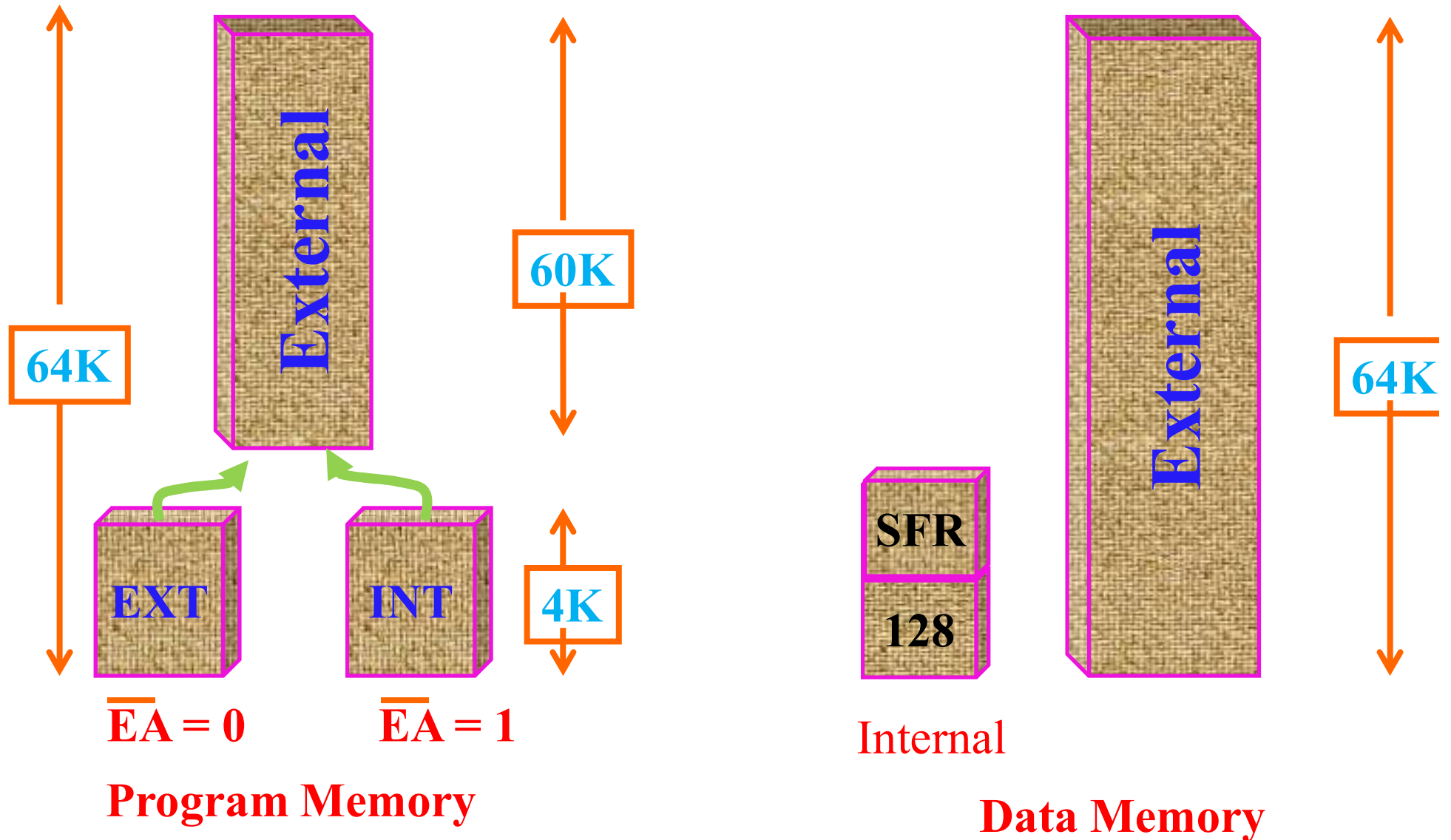
P3 Bit	Function	Pin	
P3.0	RxD	10	{ Serial communications
P3.1	TxD	11	
P3.2	$\overline{\text{INT0}}$	12	{ External interrupts
P3.3	$\overline{\text{INT1}}$	13	
P3.4	T0	14	{ Timers
P3.5	T1	15	
P3.6	$\overline{\text{WR}}$	16	{ Read/Write signals of external memories
P3.7	$\overline{\text{RD}}$	17	

General Block Diagram of 8051



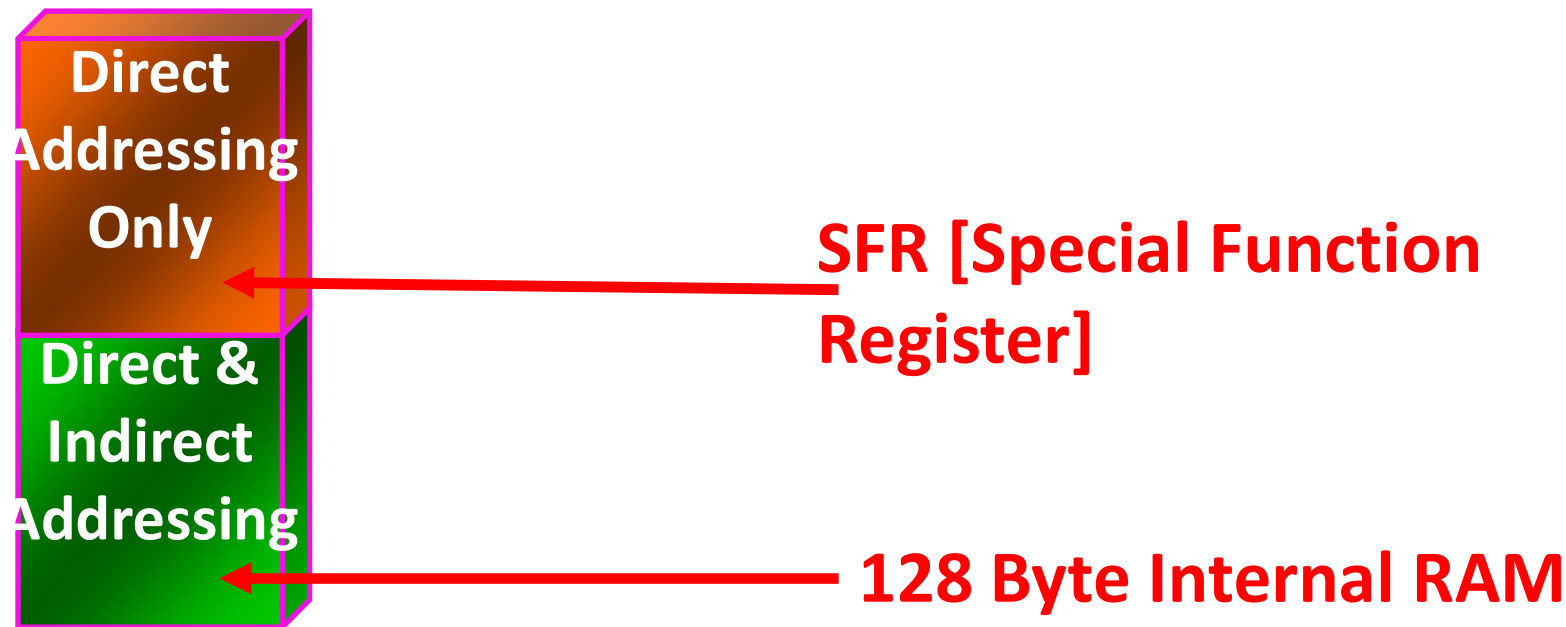


8051 Memory Organization



Internal Data Memory

- ⌘ Internal data memory contains all the processor state
 - ⏏ Lower 128 bytes: registers, general data
 - ⏏ Upper 128 bytes: special function registers



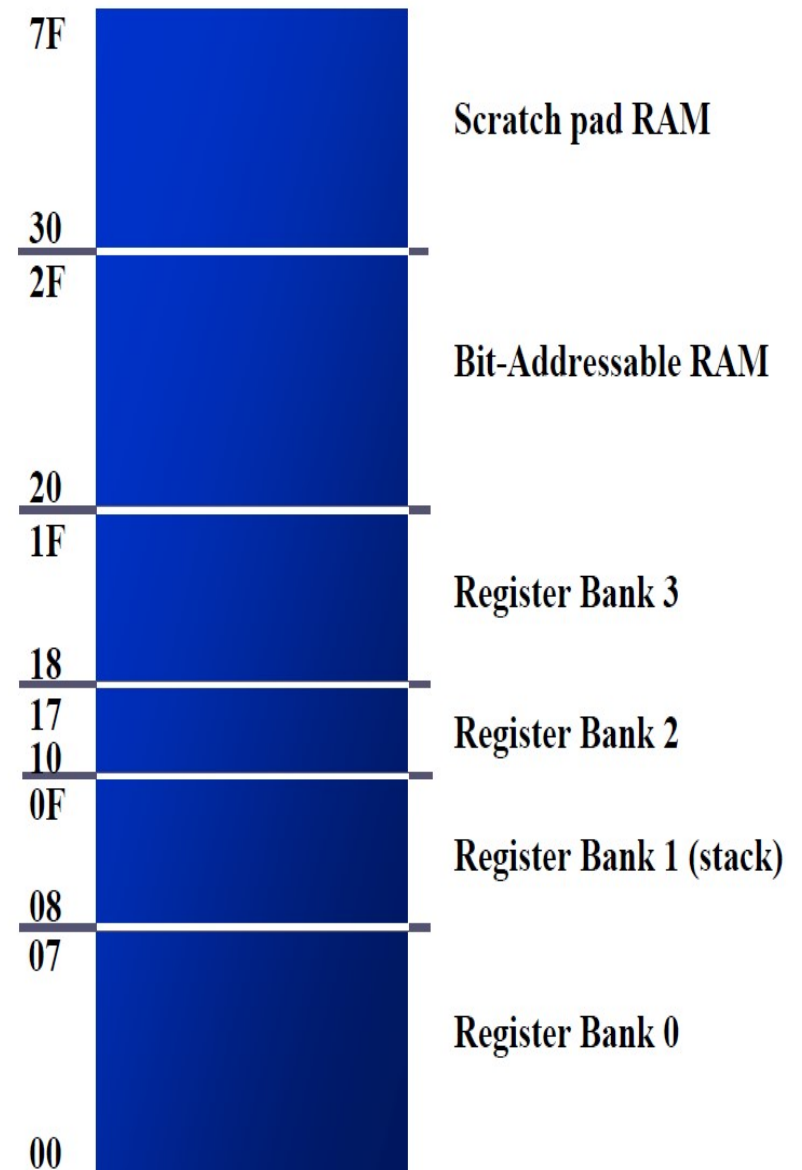
128 Byte RAM

⌘ There are 128 bytes of RAM in the 8051.

☑ Assigned addresses 00 to 7FH

⌘ The **128 bytes are divided into 3 different groups** as follows:

1. A total of **32 bytes** from locations 00 to 1F hex are set aside for **register banks** and the **stack**.
2. A total of **16 bytes** from locations 20H to 2FH are set aside for **bit-addressable** read/write memory.
3. A total of **80 bytes** from locations 30H to 7FH are used for read and write storage, called **scratch pad**.



8051 Register Banks with address

Register
bank 0

00	R0
01	R1
02	R2
03	R3
04	R4
05	R5
06	R6
07	R7

Register
bank 1

08	R0
09	R1
0A	R2
0B	R3
0C	R4
0D	R5
0E	R6
0F	R7

Register
bank 2

10	R0
11	R1
12	R2
13	R3
14	R4
15	R5
16	R6
17	R7

Register
bank 3

18	R0
19	R1
1A	R2
1B	R3
1C	R4
1D	R5
1E	R6
1F	R7

Registers in 80C51



⌘ General Purpose Register-

- ☒ Accumulator(A)-8bit
- ☒ B-register(8-bit)-for multiplication & division operation
- ☒ Program counter(16 bit)
- ☒ Stack Pointer (8 bit)
- ☒ Register R0-R7(8 bit)

Bank 0-00-07H

Bank 1-08-0FH

Bank 2-10-17H

Bank 3-18-1FH

Program Status Word

PSW: PROGRAM STATUS WORD. BIT ADDRESSABLE.

CY	AC	F0	RS1	RS0	OV	—	P
----	----	----	-----	-----	----	---	---

CY	PSW.7	Carry Flag.
AC	PSW.6	Auxiliary Carry Flag.
F0	PSW.5	Flag 0 available to the user for general purpose.
RS1	PSW.4	Register Bank selector bit 1 (SEE NOTE 1).
RS0	PSW.3	Register Bank selector bit 0 (SEE NOTE 1).
OV	PSW.2	Overflow Flag.
—	PSW.1	User definable flag.
P	PSW.0	Parity flag. Set/cleared by hardware each instruction cycle to indicate an odd/even number of '1' bits in the accumulator.

RS1	RS0	Register Bank	Address
0	0	0	00H-07H
0	1	1	08H-0FH
1	0	2	10H-17H
1	1	3	18H-1FH

Addressing modes

- ⌘ Register addressing mode-MOV A,R7
- ⌘ Direct addressing mode-MOV A,54H
- ⌘ Register Indirect addressing mode-MOV A,@R1
- ⌘ Immediate addressing mode-MOV A,#45H
- ⌘ Base Register plus Index Register Indirect addressing mode-use to access program memory.

MOVC A,@A+DPTR

☒ DPTR=0fff0H

☒ A=05

☒ Add=0fff0+05=0fff5H

SFR Registers & their Addresses



MOV 0E0H,#55H ;is the same as
MOV A,#55H ;which means load 55H into A (A=55H)

MOV 0F0H,#25H ;is the same as
MOV B,#25H ;which means load 25H into B (B=25H)

MOV 0E0H,R2 ;is the same as
MOV A,R2 ;which means copy R2 into A

MOV 0F0H,R0 ;is the same as
MOV B,R0 ;which means copy R0 into B

SFR Addresses (1 of 2)

Table 5-1: Special Function Register (SFR) Addresses

Symbol	Name	Address
ACC*	Accumulator	0E0H
B*	B register	0F0H
PSW*	Program status word	0D0H
SP	Stack pointer	81H
DPTR	Data pointer 2 bytes	
DPL	Low byte	82H
DPH	High byte	83H
P0*	Port 0	80H
P1*	Port 1	90H
P2*	Port 2	0A0H
P3*	Port 3	0B0H
IP*	Interrupt priority control	0B8H
IE*	Interrupt enable control	0A8H
TMOD	Timer/counter mode control	89H

SFR Addresses (2 of 2)

TCON*	Timer/counter control	88H
T2CON*	Timer/counter 2 control	0C8H
T2MOD	Timer/counter mode control	0C9H
TH0	Timer/counter 0 high byte	8CH
TL0	Timer/counter 0 low byte	8AH
TH1	Timer/counter 1 high byte	8DH
TL1	Timer/counter 1 low byte	8BH
TH2	Timer/counter 2 high byte	0CDH
TL2	Timer/counter 2 low byte	0CCH
RCAP2H	T/C 2 capture register high byte	0CBH
RCAP2L	T/C 2 capture register low byte	0CAH
SCON*	Serial control	98H
SBUF	Serial data buffer	99H
PCON	Power control	87H

* Bit addressable (discussed further in Chapter 8)

Interrupts



⌘ Allow parallel tasking

- ☒ Interrupt routine runs in "background"

⌘ 8051 Interrupts

- ☒ External interrupt 0(0003H)
- ☒ Timer 0 overflow(000BH)
- ☒ External interrupt 1(0013H)
- ☒ Timer 1 overflow(001BH)
- ☒ Serial port - wake up when data arrives/data has left(0023H)

Controlling Interrupts: Enables and Priority

(MSB)				(LSB)			
EA	X	X	ES	ET1	EX1	ET0	EX0

Symbol	Position	Function
EA	IE.7	Disables all interrupts. If EA = 0, no interrupt will be acknowledged. If EA = 1, each interrupt source is individually enabled or disabled by setting or clearing its enable bit.
	IE.6	Reserved.
	IE.5	Reserved.
ES	IE.4	Enables or disables the Serial Port interrupt. If ES = 0, the Serial Port interrupt is disabled.
ET1	IE.3	Enables or disables the Timer 1 Overflow interrupt. If ET1 = 0, the Timer 1 interrupt is disabled.
EX1	IE.2	Enables or disables External Interrupt 1. If EX1 = 0, External Interrupt 1 is disabled.
ET0	IE.1	Enables or disables the Timer 0 Overflow interrupt. If ET0 = 0, the Timer 0 interrupt is disabled.
EX0	IE.0	Enables or disables External Interrupt 0. If EX0 = 0, External Interrupt 0 is disabled.

SU00474

Figure 17. Interrupt Enable (IE) Register

(MSB)				(LSB)			
X	X	X	PS	PT1	PX1	PT0	PX0

Symbol	Position	Function
	IP.7	Reserved.
	IP.6	Reserved.
	IP.5	Reserved.
PS	IP.4	Defines the Serial Port interrupt priority level. PS = 1 programs it to the higher priority level.
PT1	IP.3	Defines the Timer 1 interrupt priority level. PT1 = 1 programs it to the higher priority level.
PX1	IP.2	Defines the External Interrupt 1 priority level. PX1 = 1 programs it to the higher priority level.
PT0	IP.1	Enables or disables the Timer 0 Interrupt priority level. PT0 = 1 programs it to the higher priority level.
PX0	IP.0	Defines the External Interrupt 0 priority level. PX0 = 1 programs it to the higher priority level.

SU00475

Figure 18. Interrupt Priority (IP) Register

Timer/Counter



- ⌘ 8051 has 2 timers
- ⌘ Timer mode
 - ☒ Increments every machine cycle (12 clock cycles)
- ⌘ Counter mode
 - ☒ Increments when T0/T1 go from 1 - 0 (external signal)
- ⌘ Timer can cause an interrupt
- ⌘ Timer/Counter operation
 - ☒ Mode control register (TMOD)
 - ☒ Control register (TCON)

Timer/Counter Control Register (TCON)

- ⌘ TR - enable timer/counter
- ⌘ TF - overflow flag: can cause interrupt
- ⌘ IE/IT - external interrupts and type control
 - ☒ not related to the timer/counter

MSB
LSB

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

BIT	SYMBOL	FUNCTION
TCON.7	TF1	Timer 1 overflow flag. Set by hardware on Timer/Counter overflow. Cleared by hardware when processor vectors to interrupt routine, or clearing the bit in software.
TCON.6	TR1	Timer 1 Run control bit. Set/cleared by software to turn Timer/Counter on/off.
TCON.5	TF0	Timer 0 overflow flag. Set by hardware on Timer/Counter overflow. Cleared by hardware when processor vectors to interrupt routine, or by clearing the bit in software.
TCON.4	TR0	Timer 0 Run control bit. Set/cleared by software to turn Timer/Counter on/off.
TCON.3	IE1	Interrupt 1 Edge flag. Set by hardware when external interrupt edge detected. Cleared when interrupt processed.
TCON.2	IT1	Interrupt 1 type control bit. Set/cleared by software to specify falling edge/low level triggered external interrupts.
TCON.1	IE0	Interrupt 0 Edge flag. Set by hardware when external interrupt edge detected. Cleared when interrupt processed.
TCON.0	IT0	Interrupt 0 Type control bit. Set/cleared by software to specify falling edge/low level triggered external interrupts.

SU00536

Figure 8. Timer/Counter Control (TCON) Register

Mode Control Register (TMOD)

- ⌘ **GATE** - allows external pin to enable timer (e.g. external pulse)
 - ☒ 0: INT pin not used
 - ☒ 1: counter enabled by INT pin (port 3.2, 3.3)
- ⌘ **C/T** - indicates timer or counter mode
 - ☒ 0-timer
 - ☒ 1-counter

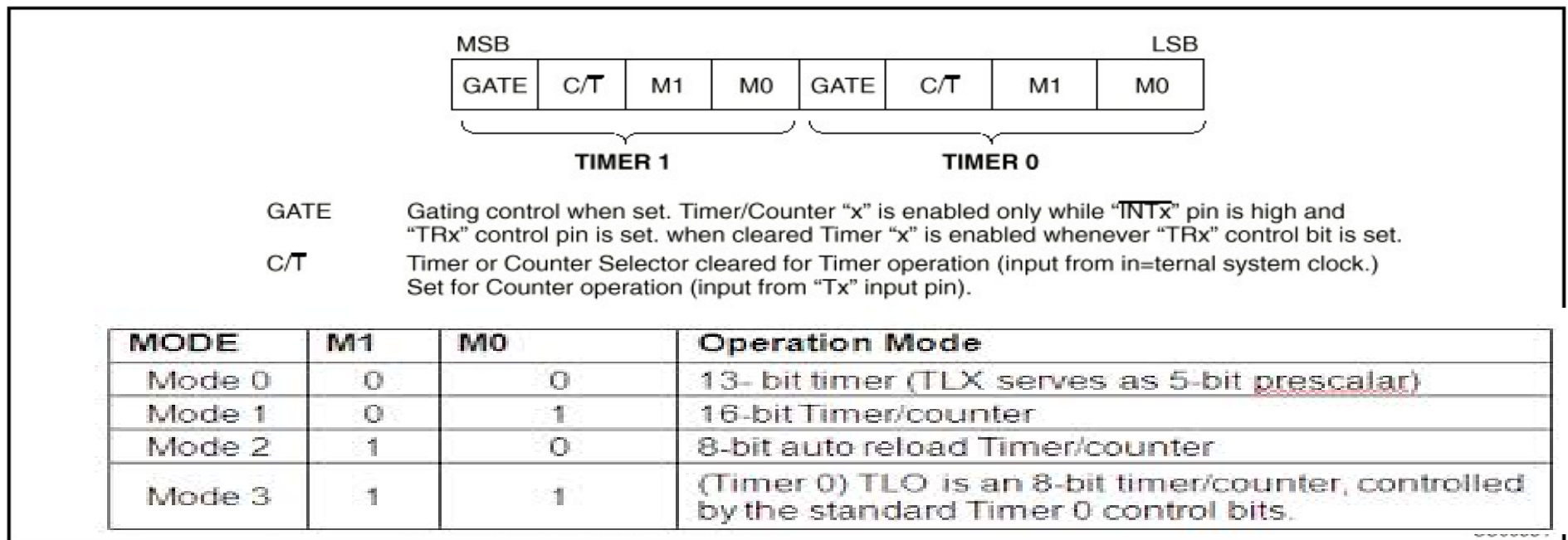
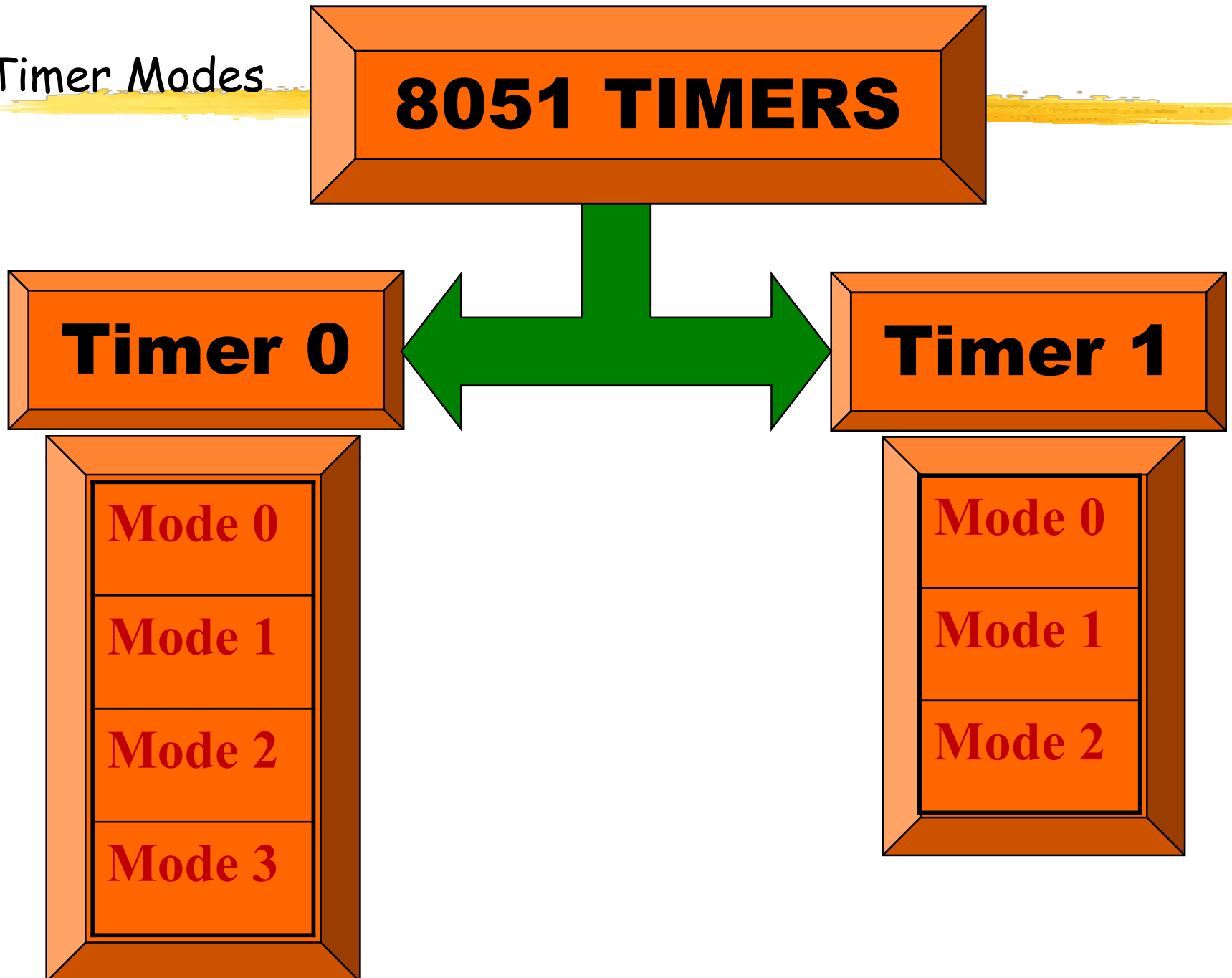


Figure 6. Timer/Counter Mode Control (TMOD) Register

8051 Timer Modes



Timer/Counter Mode 0 & 1

- ⌘ Mode 0 is 13 bits timer/counter with 5bit TLO or TL1 as prescaler and complete TH0 or TH1
- ⌘ Mode 1 same as Mode 0, but uses all 16 bits.

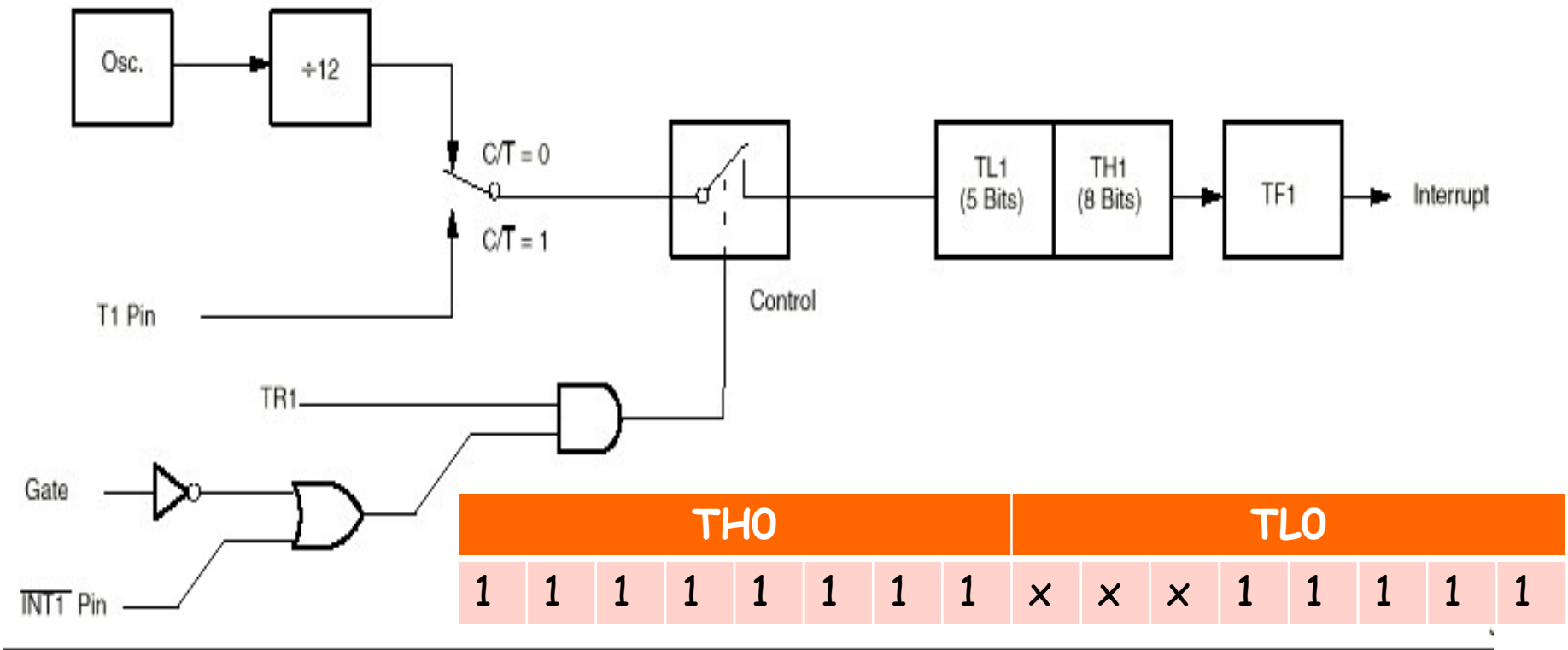


Figure 7. Timer/Counter Mode 0: 13-Bit Counter

Timer/Counter Mode 2

⌘ 8-bit counter, auto-reload on overflow

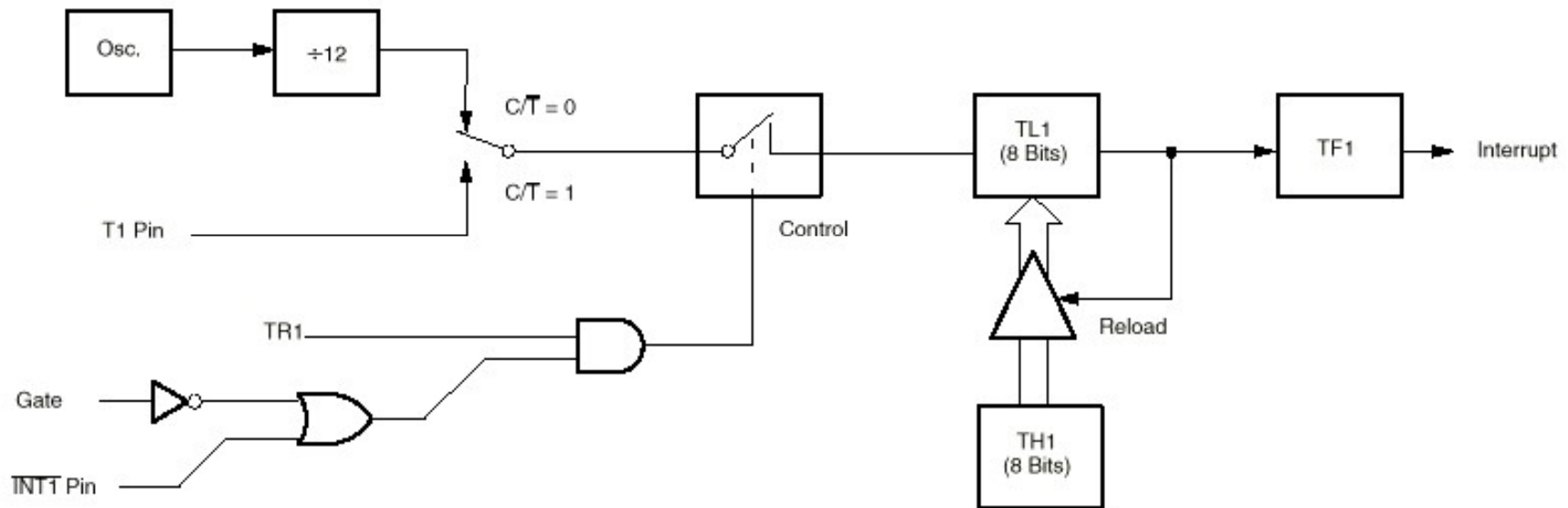


Figure 9. Timer/Counter Mode 2: 8-Bit Auto-Load

Timer/Counter Mode 3

- ⌘ Applies to Timer/Counter 0
- ⌘ Gives an extra timer

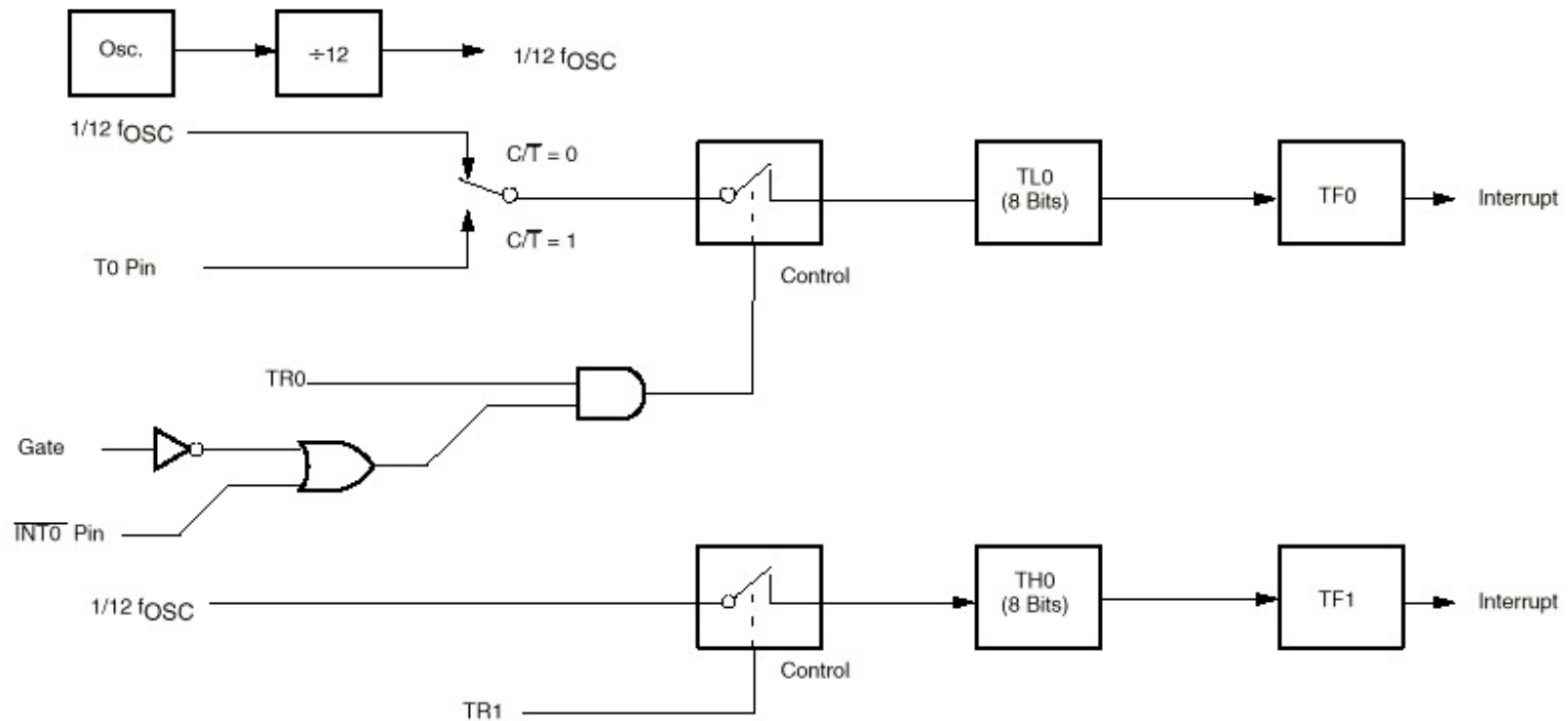


Figure 10. Timer/Counter 0 Mode 3: Two 8-Bit Counters

Programming Timers

⌘ **Example:** Indicate which mode and which timer are selected for each of the following.

(a) `MOV TMOD, #01H` (b) `MOV TMOD, #20H` (c) `MOV TMOD, #12H`

⌘ **Solution:** We convert the value from hex to binary.

(a) `TMOD = 00000001`, mode 1 of timer 0 is selected.

(b) `TMOD = 00100000`, mode 2 of timer 1 is selected.

(c) `TMOD = 00010010`, mode 2 of timer 0, and mode 1 of timer 1 are selected.

Programming Timers

- ⌘ Find the timer's clock frequency and its period for various 8051-based system, with the crystal frequency 11.0592 MHz when C/T bit of TMOD is 0.

⌘ **Solution:**



$$1/12 \times 11.0529 \text{ MHz} = 921.6 \text{ MHz};$$

$$T = 1/921.6 \text{ kHz} = 1.085 \text{ us}$$

Serial Communication



- ⌘ 8051 supports a full duplex serial port.
- ⌘ RXD and TXD pins are used for serial communication.
- ⌘ Supports RS232 standard.

- ⌘ Baud rate: is the reciprocal of the time to send 1 bit.
- ⌘ Each byte is preceded by a start bit and one stop bit.
- ⌘ Start and stop bit is used to synchronize the serial receiver.
- ⌘ The data byte is always transmitted with least significant bit (LSB) first.
- ⌘ For error checking it is possible to include a parity bit as well, just prior to the stop bit.
- ⌘ SBUF register is used to store data for serial communication.

Serial Port Control Register (SCON)

SCON.7	SCON.6	SCON.5	SCON.4	SCON.3	SCON.2	SCON.1	SCON.0
SM0	SM1	SM2	REN	TB8	RB8	TI	RI

SCON bit	Description
SM0	Serial communication Mode
SM1	
SM2	In mode 2 and 3, if set, this will enable multiprocessor communication.
REN	(Receive enable) Enable serial reception
TB8	This is the 9 th data bit that is transmitted in mode 2 and 3.
RB8	9 th data bit that is received in mode 2 and 3. it is not used in mode 0. In mode 1, if SM2=0 then RB8 is the stop bit that is received.
TI	Transmit interrupt flag, set by hardware must be cleared by software.
RI	Receive interrupt flag, set by hardware must be cleared by software.



SM0	SM1	Mode	Description	Baud rate
0	0	0	8 bit shift register mode	Fosc/12
0	1	1	8 bit UART	Variable (set by timer 1)
1	0	2	9 bit UART	Fosc/32 or fosc/64
1	1	3	9 bit UART	Variable (set by timer 1)

Mode 0 (shift Register Mode)

- ⌘ Only RXD is pin through which data enters or exits.
- ⌘ TXD pins output the shift clock only.
- ⌘ Eight bit data are transmitted or received.
- ⌘ Baud rate is fixed ($f_{osc}/12$). Total determine by system clock frequency (f_{osc}).



Mode 1

- ⌘ 10 bits are transmitted through TXD pin or received through RXD pin.
- ⌘ Start bit (0), then 8 bit data (LSB first) and a stop bit (1).
- ⌘ Baud rate is variable and determined by timer 1 overflow rate.
- ⌘ Baud rate=variable (depends on timer 1 overflow rate)



Mode 2

- ⌘ 11 bits are transmitted with a '0' start bit, then 8 data bit, a 9th bit and a stop bit '1'.
- ⌘ 9th bit is programmable,
- ⌘ User program can define 9th bit as TB8 in SCON. It may be parity of data byte.
- ⌘ Baud rate = $f_{osc}/32$ OR $f_{osc}/64$



Mode 3



- ⌘ Same as mode 2 except that the baud rate is defined by timer 1 overflow rate.



THANK YOU